



Compilers

Guillermo A. Pérez

Lecture 6: Bottom-up parsing with look-ahead

TL;DR: This lecture in short

How do we go from grammars to a deterministic parser?

We will try to do so by applying the grammar rules **in reverse** and using look-ahead.

Main references

- **Introduction to Language Theory and Compilers.** Gilles Geeraerts and G. A. Pérez. 2019.
- **Crafting a Compiler.** Charles N. Fischer et al. 2010. Pearson Education.
- **Compiler Design.** Helmut Seidl et al. 2012. Springer
- **Compilers: Principles, Techniques, Tools.** Alfred Aho et al. 2006. Pearson Education.
- Online: Wikipedia, LLVM, ANTLR, ...

Required and target competences

What tools do we need?

Formal language theory (languages and machines, machines and computability); **THE PREVIOUS LECTURE!**

What skills will we obtain?

- theory: canonical finite automaton with look-ahead
- practice: how to implement SLR and LR

How will these skills be useful?

- It will allow you to understand the implementation of (older/other) parser generators.

Outline

1. SLR(k) parsers
2. LR(k) parsers
3. A hierarchy of CFLs

SLR(k) parsers

Why more parsers?

There are some grammars which do not yield deterministic LR(0) parsers, i.e. they are not LR(0).^a We will add look-ahead, **in ever more complicated ways**, to our parsers

- to make them deterministic.

^aAn example follows

What are they?

They are **Simple Left scanning, Right parsing** parsers with k -look-ahead.

Reminder: First and Follow

Definition (First)

For α a sentential form, we define

$$\text{First}^k(\alpha) := \{w \in T^* : \alpha \Rightarrow^* wx \wedge (|w| = k \text{ or } |w| < k \wedge x = \varepsilon)\}.$$

Definition (Follow)

For α a sentential form, we define

$$\text{Follow}^k(\alpha) := \{w \in T^* \mid \exists \beta, \gamma : S \Rightarrow^* \beta\alpha\gamma \wedge w \in \text{First}^k(\gamma)\}.$$

The action table for SLR parsers

The action table

Index grammar rules $1 \leq j \leq n$ and states from the CFSM $0 \leq i \leq k$. The table T maps each i to a set of actions:

- $T(i)$ contains j if state i has the item $A \rightarrow \alpha \bullet$ with $A \rightarrow \alpha$ the j -th rule.
- $T(i)$ contains (a Shift) a if state i has an item $A \rightarrow \alpha_1 \bullet a\alpha_2$.
- $T(i)$ contains Accept if state i has an item $S \rightarrow \alpha \bullet$.
- $T(\emptyset)$ contains an Error action only.

Action table with look-ahead

Algorithm

1. Push the initial state 0 of the CFSM into the stack $\mathcal{S}$.
2. As long as we can't accept or get an error:
 - 2.1 If $j \in T(\text{top}(\mathcal{S}))$, $A \rightarrow \alpha$ is the j -th rule, and the look-ahead ℓ is in $\text{Follow}^k(A)$, then
 - 2.1.1 pop $|\alpha|$ times from $\mathcal{S}$
 - 2.1.2 and put A in variable c .
 - 2.2 Otherwise, if the look-ahead starts with a , put a into the variable c .
 - 2.3 Let q' be the next state $\delta(\text{top}(\mathcal{S}), c)$ of the CFSM after reading c .
 - 2.4 Push q' into $\mathcal{S}$.

Notes on getting a real table, not a vector

The Follow^k sets can be pre-computed: the look-ahead combinations from those sets become columns.

- The parser is deterministic if ≤ 1 entry per cell
- The grammar is then SLR(k)

Exercise: SLR(1) grammar

(1)	Exp	→	Exp + Prod
(2)		→	Prod
(3)	Prod	→	Prod * Atom
(4)		→	Atom
(5)	Atom	→	Cst
(6)		→	Id

- Build the LR(0) action table for the grammar. Is it an LR(0) grammar?
- Build the SLR(1) action table for the grammar. Is it an SLR(1) grammar?

General LR(k) parsers, as Knuth intended

A real table

We described the case of $k = 0$ only for LR parsers. T was then a **vector**! If we have k -look-ahead, then it maps states and look-ahead words to actions.

- Instead of using the same CFSM and hack it with extra information. . .
- We generalize the CFSM to a LR(k)-CFSM.
- This means we need to re-visit the notions of item, closure, and successor.

Items with look-ahead

Definition (LR(k)-items)

These are item-look-ahead pairs $(A \rightarrow \alpha_1 \bullet \alpha_2, w)$ where $w \in \Sigma^{\leq k}$.

Definition (LR(k)-closure)

The closure of an LR(k)-item $(A \rightarrow \alpha_1 \bullet B\alpha_2, w)$ is the set of all LR(k)-items $(B \rightarrow \bullet\beta, y)$ where

- $B \rightarrow \beta$ is a rule of the grammar, and
- $y \in \text{First}^k(\alpha_2 w)$.

As before, the closure of a set I is the minimal closure-closed set including I .

The LR(k)-CFSM

Definition (CFSM states)

States of the CFSM are now subsets of LR(k)-items. The a -successor of a state I , for $a \in T \cup V$, is the closure of the set

$$\{(A \rightarrow \alpha_1 a \bullet \alpha_2, w) \mid (A \rightarrow \alpha_1 \bullet a \alpha_2, w) \in I\} .$$

Definition (The LR(k)-CFSM)

It is the DFA $(Q, V \cup T, \delta, q_0)$ where

- Q is the set of all subsets of LR(k)-items,
- q_0 is the closure of $\{(S \rightarrow \bullet \alpha, \varepsilon) \mid S \rightarrow \alpha \text{ is a rule}\}$,
- $\delta(q, a)$ maps to the closure of the a -successor of q .
- (All states are accepting! Except for the empty set, i.e. a bad sink.)

The action table for LR(k) parsers

The action table

Index grammar rules $1 \leq j \leq n$ and states from the CFSM $0 \leq i \leq k$. The table T maps each i and look-ahead ℓ to a set of actions:

- $T(i, \ell)$ contains (a Reduce) j if state i has the item $(A \rightarrow \alpha \bullet, \ell)$ with $A \rightarrow \alpha$ the j -th rule.
- $T(i, \ell)$ contains Shift if state i has an item $(A \rightarrow \alpha_1 \bullet \alpha_2, y)$ and $\ell \in \text{First}^k(\alpha_2 y)$.
- $T(i, \varepsilon)$ contains Accept if state i has an item $S \rightarrow \alpha \bullet$.
- $T(\emptyset, \ell)$ contains an Error action only.

Deterministic

- The parser is deterministic if every cell has exactly one entry.
- If the latter holds, the grammar is LR(k).

Exercise: LR(1) grammar

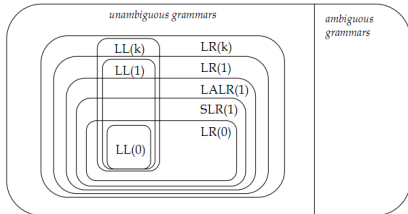
(1)	S	$\rightarrow$	$S'\$$
(2)	S'	$\rightarrow$	(M)
(3)		$\rightarrow$	$\{M\}$
(4)		$\rightarrow$	(U)
(5)		$\rightarrow$	$\{U\}$
(6)	U	$\rightarrow$	Expr
(7)	M	$\rightarrow$	Expr

- Build the LR(1) action table for the grammar. Is it an LR(1) grammar?

Conclusions

LL(1) versus LR(k)

A picture is worth a thousand words:



Theorem

For all $k \geq 1$, the $LR(k)$ languages, $LR(1)$ languages, and languages recognizable by deterministic PDAs are all the same.

Note

Most widely-used programming languages have an $LALR(1)$ grammar.